

Handwritten Digit Recognition Using Convolutional Neural Networks

Major Project Report

Submitted by

Nirupama Sadunapalli

Submitted to

Human Resources of Maa Software Solutions
@hrmaasoftwaresolutions.in

July 20,2026

Abstract

Handwritten Digit Recognition is a fundamental application of Deep Learning and Computer Vision. This project presents a Convolutional Neural Network (CNN) model trained on the MNIST dataset to recognize handwritten digits from 0 to 9. The model automatically extracts features through convolutional layers and classifies the input image with high accuracy. A Streamlit-based interface allows users to draw digits and obtain real-time predictions. Experimental results demonstrate that CNN provides reliable performance and is suitable for practical character recognition applications.

Contents

Abstract	1
1 Introduction	3
2 Literature Survey	5
3 Problem Statement	8
4 Objectives	10
5 System Requirements	12
6 Dataset Description	14
7 CNN Architecture	18
8 Methodology and Algorithms	26
9 Results	34
10 Applications and Future Scope	38
11 Conclusion	40
12 References	41

Introduction

Handwritten Digit Recognition (HDR) is an important application of Artificial Intelligence (AI), Machine Learning (ML), and Computer Vision. It involves automatically identifying handwritten numerical digits from digital images. Although humans recognize handwritten digits effortlessly, computers face challenges due to variations in handwriting style, size, orientation, stroke thickness, and image quality.

HDR has gained significant attention because of its applications in postal automation, bank cheque processing, document digitization, form recognition, educational systems, and mobile applications. Automated recognition systems reduce manual effort, improve efficiency, and minimize human errors.

Earlier handwritten digit recognition systems relied on traditional machine learning algorithms such as Support Vector Machines (SVM), Decision Trees, Random Forests, and k-Nearest Neighbors (k-NN). These methods required manual feature extraction using techniques such as edge detection, gradients, and geometric descriptors. Although effective for simple datasets, their performance depended heavily on handcrafted features and was limited when handling complex handwriting variations.

The emergence of Deep Learning has transformed image classification by enabling automatic feature extraction from raw image data. Among various deep learning models, Convolutional Neural Networks (CNNs) have become the preferred choice for handwritten digit recognition. CNNs automatically learn hierarchical features ranging from edges and curves to complete digit shapes, resulting in significantly higher recognition accuracy.

This project uses the widely accepted MNIST dataset, which contains 70,000 grayscale images of handwritten digits (60,000 training and 10,000 testing images). Each image has a resolution of 28×28 pixels, making the dataset an ideal benchmark for evaluating handwritten digit recognition models.

A CNN model is developed using TensorFlow and Keras to classify handwritten digits into ten classes (0–9). The architecture consists of convolutional, pooling, batch normalization, and fully connected layers trained using the Adam optimizer and categorical cross-entropy loss function. To demonstrate practical implementation, the trained model

is integrated with a Streamlit web application that allows users to draw handwritten digits and obtain real-time predictions after appropriate image preprocessing.

This report presents the complete development process of the proposed system, including dataset preparation, CNN architecture, preprocessing techniques, model training, performance evaluation, and deployment. The obtained results demonstrate that CNNs provide highly accurate and reliable handwritten digit recognition suitable for real-world applications.

1.1 Motivation

The increasing demand for intelligent document processing has created a need for automated handwritten digit recognition systems. Manual processing is time-consuming, expensive, and prone to errors. CNN-based recognition systems provide high accuracy, fast prediction, and minimal human intervention, making them suitable for modern computer vision applications.

This project aims to understand the practical implementation of CNNs while developing an efficient real-time handwritten digit recognition system.

1.2 Project Overview

The proposed system consists of two major stages:

1. Development and training of a Convolutional Neural Network using the MNIST handwritten digit dataset.
2. Deployment of the trained CNN model through a Streamlit web application for real-time digit prediction.

The workflow begins with loading and preprocessing the MNIST dataset, followed by CNN model training and evaluation. After achieving satisfactory accuracy, the trained model is integrated into a Streamlit application that enables users to draw handwritten digits and receive instant predictions along with confidence scores.

The project demonstrates the effectiveness of Convolutional Neural Networks in solving handwritten digit recognition problems while providing an interactive and user-friendly deployment platform.

Literature Survey

Handwritten Digit Recognition (HDR) has been an active area of research in pattern recognition, machine learning, and computer vision for several decades. Researchers have proposed numerous techniques to improve recognition accuracy, computational efficiency, and robustness to variations in handwriting styles. The evolution of handwritten digit recognition can be broadly divided into traditional machine learning approaches and modern deep learning approaches.

2.1 Early Research

Early handwritten digit recognition systems relied on handcrafted feature extraction. Researchers extracted geometric features, zoning information, edge maps, projection histograms, chain codes, and statistical descriptors before applying classifiers such as Decision Trees and Bayesian classifiers.

2.2 Machine Learning-Based Methods

2.2.1 K-Nearest Neighbors (KNN)

KNN compares a test sample with its nearest neighbors. It is simple but computationally expensive for large datasets.

2.2.2 Support Vector Machine (SVM)

SVM provides strong classification performance but depends on carefully engineered input features.

2.2.3 Artificial Neural Networks (ANN)

ANNs improved recognition accuracy but flattened image inputs, losing important spatial information.

2.3 Deep Learning Approaches

Deep learning automatically learns features from raw images. Convolutional Neural Networks (CNNs) became the dominant approach because they preserve spatial relationships while learning hierarchical image features.

2.4 Convolutional Neural Networks

CNNs consist of convolutional layers, activation functions, pooling layers, and fully connected layers. LeNet-5 demonstrated the effectiveness of CNNs for handwritten character recognition and inspired many modern architectures.

2.5 MNIST Dataset

The MNIST dataset contains 70,000 grayscale images of handwritten digits (60,000 for training and 10,000 for testing). It is the standard benchmark for handwritten digit recognition.

2.6 Comparison of Existing Techniques

Table 2.1: Comparison of Existing Methods

Method	Accuracy	Limitation
KNN	Moderate	Slow prediction
SVM	High	Feature engineering
ANN	High	Loses spatial information
CNN	Very High	Higher computation

2.7 Research Gap

Traditional methods require handcrafted features and are less robust to handwriting variations. CNNs overcome these issues through automatic feature learning and improved generalization.

2.8 Summary

The literature indicates that CNN-based models provide the best balance of accuracy, robustness, and scalability for handwritten digit recognition. Therefore, this project adopts a CNN trained on the MNIST dataset and deploys it using a Streamlit application for real-time prediction.

Problem Statement

Handwritten digit recognition is a challenging computer vision task due to variations in handwriting style, size, orientation, stroke thickness, and image quality. These variations make it difficult for traditional machine learning methods, which rely on manual feature extraction and often fail to capture complex image patterns effectively.

Accurate handwritten digit recognition is essential in applications such as cheque processing, postal code recognition, document digitization, form processing, and educational systems. Manual processing of handwritten documents is time-consuming, costly, and prone to human error, creating the need for an efficient automated solution.

Recent advances in deep learning, particularly Convolutional Neural Networks (CNNs), have significantly improved image classification performance. CNNs automatically learn meaningful features from raw image data, eliminating manual feature engineering and providing higher accuracy than conventional methods.

In this project, a CNN-based handwritten digit recognition system is developed using the MNIST dataset to classify digits (0–9). The trained model is integrated with a Streamlit web application that enables users to draw handwritten digits and receive real-time predictions with confidence scores. The proposed system aims to provide an accurate, efficient, and user-friendly solution for handwritten digit recognition.

3.1 Need for the Proposed System

The proposed system is designed to overcome the limitations of conventional handwritten digit recognition methods by:

- Automatically extracting image features using Convolutional Neural Networks.
- Improving recognition accuracy for diverse handwriting styles.
- Reducing manual effort and processing time.
- Providing real-time handwritten digit prediction through a web application.

- Delivering high classification performance with efficient computation.
- Offering a scalable framework that can be extended to handwritten character recognition, Optical Character Recognition (OCR), and document digitization.

3.2 Scope of the Project

The scope of this project includes the development of a deep learning model capable of recognizing handwritten digits from grayscale images using the MNIST dataset. The project focuses on image preprocessing, CNN model training, model evaluation, and deployment through a Streamlit application.

The system is limited to recognizing decimal digits (0–9). However, the overall framework can be extended in the future to recognize handwritten alphabets, mathematical symbols, multilingual scripts, and complete handwritten documents using more advanced deep learning architectures.

Objectives

The primary objective of this project is to design, develop, and evaluate a handwritten digit recognition system using Convolutional Neural Networks (CNNs). The system aims to accurately recognize handwritten digits (0–9) from the MNIST dataset and provide real-time predictions through a Streamlit web application.

The specific objectives of the project are:

1. To understand the fundamentals of Artificial Intelligence, Machine Learning, Deep Learning, and Convolutional Neural Networks.
2. To study the MNIST dataset and perform preprocessing techniques such as normalization, reshaping, and grayscale image preparation.
3. To design and implement an efficient CNN architecture for automatic handwritten digit recognition.
4. To train and optimize the CNN model using TensorFlow, Keras, the Adam optimizer, and categorical cross-entropy loss.
5. To evaluate the model using performance metrics such as accuracy, loss, confusion matrix, and classification accuracy.
6. To develop a Streamlit-based web application for real-time handwritten digit prediction.
7. To demonstrate the effectiveness of CNNs over traditional machine learning techniques for image classification.
8. To develop a scalable system that can be extended to Optical Character Recognition (OCR), handwritten alphabet recognition, and multilingual character recognition.

4.1 Expected Outcomes

The expected outcomes of this project include:

- A robust CNN model capable of accurately recognizing handwritten digits.
- High classification accuracy on the MNIST dataset.
- Automatic feature extraction without manual feature engineering.
- Fast and reliable real-time prediction through a Streamlit interface.
- A scalable framework suitable for future handwritten character recognition and OCR applications.

System Requirements

The successful implementation of the proposed handwritten digit recognition system requires appropriate hardware and software resources.

5.1 Hardware Requirements

Table 5.1: Hardware Requirements

Component	Specification
Processor	Intel Core i5 (8th Gen or above) / AMD Ryzen 5
RAM	Minimum 8 GB (16 GB Recommended)
Storage	20 GB Free Disk Space
Graphics	Integrated/NVIDIA GPU (Optional)
Display	1366x768 or higher

5.2 Software Requirements

Table 5.2: Software Requirements

Operating System	Windows 10/11, Ubuntu, macOS
Language	Python 3.10+
IDE	VS Code / Jupyter Notebook
Framework	TensorFlow, Keras
Libraries	NumPy, Pandas, Matplotlib, OpenCV, Streamlit
Dataset	MNIST

5.3 Development Environment

Python, TensorFlow and Keras were used to develop the CNN model. Jupyter Notebook was used for training and Visual Studio Code for deployment. Streamlit provides a web interface for real-time handwritten digit prediction.

5.4 Software Libraries

- TensorFlow
- Keras
- NumPy
- Pandas
- Matplotlib
- OpenCV
- Streamlit

5.5 System Configuration

The model is trained on the MNIST dataset (60,000 training and 10,000 testing images). Images are normalized and reshaped before training. The trained model is integrated with a Streamlit application for real-time prediction.

5.6 Summary

The selected hardware and software environment provides a stable platform for developing, training and deploying the handwritten digit recognition system.

Dataset Description

The performance of a deep learning model largely depends on the quality of the dataset used for training and evaluation. In this project, the **Modified National Institute of Standards and Technology (MNIST)** dataset is used to train and test the proposed Convolutional Neural Network (CNN). MNIST is one of the most widely used benchmark datasets for handwritten digit recognition and computer vision research.

The dataset consists of grayscale images of handwritten digits from **0 to 9**. Each image has a resolution of **28×28 pixels**, making it computationally efficient while providing sufficient information for accurate classification.

6.1 Overview of the MNIST Dataset

The MNIST dataset was created from the original NIST handwritten digit database and introduced by Yann LeCun and his collaborators. It contains **70,000 handwritten digit images**, including **60,000 training images** and **10,000 testing images**. The testing set is completely unseen during training, enabling an unbiased evaluation of the CNN model.

6.2 Dataset Characteristics

The main characteristics of the MNIST dataset are summarized in Figure

Characteristics of the MNIST Dataset

Attribute	Description
 Dataset Name	MNIST
 Image Type	Grayscale
 Image Size	28 x 28 Pixels
 Number of Classes	10 (Digits 0--9)
 Training Images	60,000
 Testing Images	10,000
 Total Images	70,000
 Pixel Value Range	0--255
 Normalized Range	0.0--1.0

Made with  Napkin

6.3 Data Preprocessing

Before training, the dataset is preprocessed to improve learning efficiency. The preprocessing steps include:

1. Loading the MNIST dataset.
2. Splitting it into training and testing sets.
3. Reshaping images from 28×28 to $28 \times 28 \times 1$.
4. Normalizing pixel values from 0–255 to 0–1 using

$$x = \frac{x}{255}$$

5. Converting class labels into one-hot encoded vectors.

These preprocessing steps ensure faster convergence and better model performance.

6.4 Advantages of the MNIST Dataset

The MNIST dataset is selected because it offers several advantages:

- Standard benchmark dataset for handwritten digit recognition.
- Balanced distribution of digit classes.
- Large number of labeled training samples.
- Easy integration with TensorFlow and Keras.
- Suitable for evaluating CNN-based image classification models.

6.5 Sample Dataset Images

Figure 6.5 shows sample handwritten digit images from the MNIST dataset.

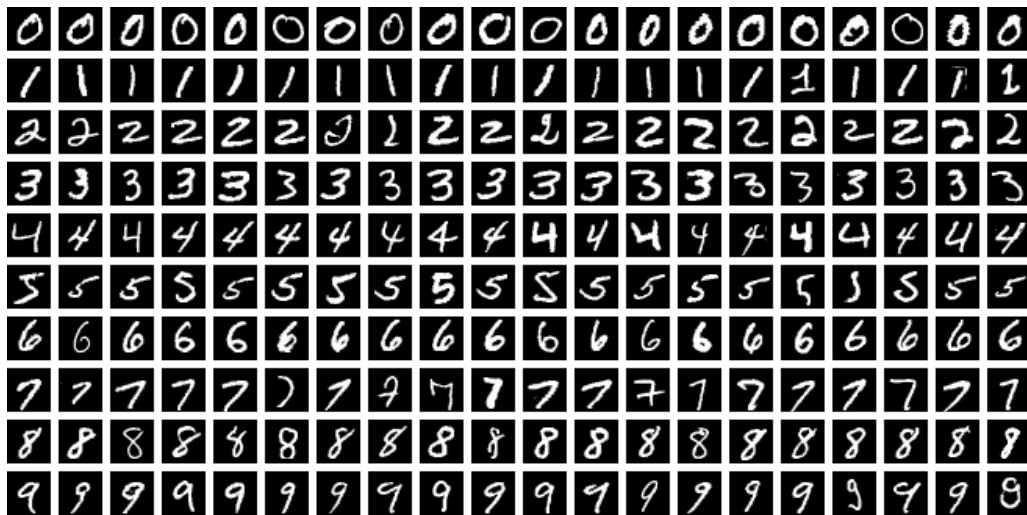


Figure 6.1: Sample Images from the MNIST Dataset

You can replace the above placeholder with sample images from the MNIST dataset.

6.6 Summary

The MNIST dataset provides a reliable benchmark for handwritten digit recognition. Its balanced class distribution, sufficient training samples, and compatibility with deep learning frameworks make it an ideal choice for developing and evaluating CNN-based models. After preprocessing, the dataset is used to train and test the proposed handwritten digit recognition system, resulting in accurate and efficient classification.

CNN Architecture

Convolutional Neural Networks (CNNs) are one of the most successful deep learning architectures for image classification and computer vision tasks. Unlike traditional machine learning algorithms, CNNs automatically learn hierarchical image features from raw pixel values without requiring manual feature extraction. Due to their high accuracy and robustness, CNNs have become the standard approach for handwritten digit recognition.

The proposed handwritten digit recognition system uses a Convolutional Neural Network implemented using TensorFlow and Keras. The model receives a grayscale image of size $28 \times 28 \times 1$ as input and predicts one of the ten digit classes (0–9). The architecture consists of convolutional layers, batch normalization, pooling layers, dropout layers, fully connected layers, and a Softmax output layer.

7.1 Architecture Overview

The CNN architecture used in this project follows a sequential design where each layer performs a specific task in the feature learning process. Initially, low-level features such as edges and curves are extracted. As the network becomes deeper, it learns more complex features such as digit shapes and patterns. Finally, the dense layers classify the extracted features into one of the ten output classes.

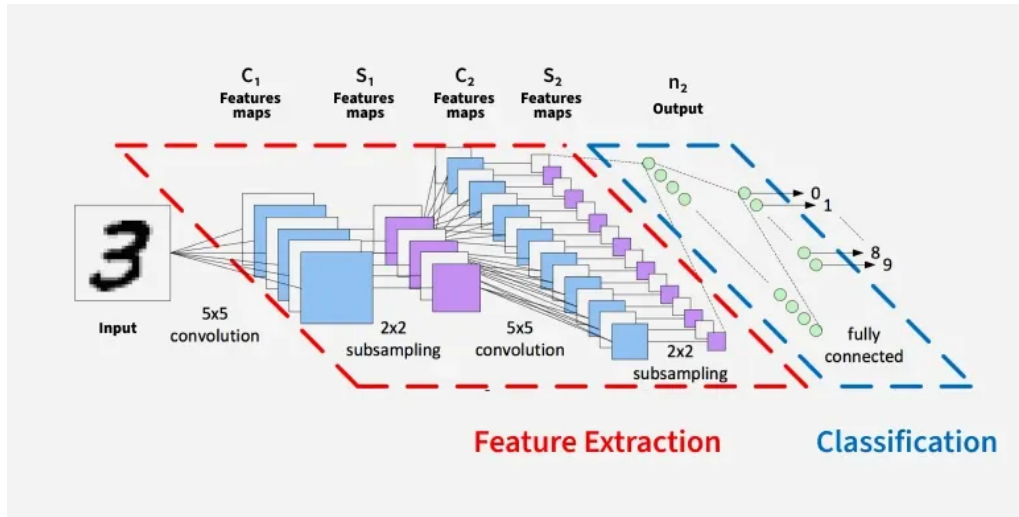


Figure 7.1: CNN Architecture

You can later replace this placeholder with the CNN architecture diagram generated using TensorFlow, Netron, draw.io, PowerPoint, or any diagram creation tool.

7.2 Working Principle of CNN

The working of a Convolutional Neural Network can be summarized in the following stages.

1. Input image acquisition
2. Convolution operation
3. Feature extraction
4. Activation using ReLU
5. Batch normalization
6. Max pooling
7. Dropout regularization
8. Flattening
9. Fully connected classification
10. Softmax prediction

Each stage contributes towards improving the learning capability and classification accuracy of the model.

7.3 Input Layer

The input layer accepts grayscale handwritten digit images of dimensions

$$28 \times 28 \times 1$$

where

- Height = 28 pixels
- Width = 28 pixels
- Channel = 1 (Grayscale)

Before feeding the images into the CNN, pixel values are normalized between 0 and 1 using

$$x' = \frac{x}{255}$$

where

- x represents the original pixel value.
- x' represents the normalized pixel value.

Normalization improves convergence during training and prevents numerical instability.

7.4 Convolution Layer

The convolution layer is the most important component of the CNN architecture. It learns image features by sliding multiple learnable filters over the input image.

Each filter detects a specific visual pattern such as edges, curves, corners, or strokes. During training, these filters automatically adjust their weights to learn the most useful image features.

The mathematical representation of convolution is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) \times K(m, n)$$

where

- X is the input image.
- K is the convolution kernel.

- Y is the resulting feature map.

The proposed CNN uses multiple convolutional layers to learn increasingly complex features from handwritten digit images.

7.5 Activation Function (ReLU)

After convolution, the Rectified Linear Unit (ReLU) activation function is applied.

The ReLU function is defined as

$$f(x) = \max(0, x)$$

Negative values are converted into zero, while positive values remain unchanged.

The advantages of ReLU include

- Faster convergence
- Reduced vanishing gradient problem
- Sparse activation
- Computational efficiency

Because of these advantages, ReLU has become the standard activation function for most CNN architectures.

7.6 Batch Normalization

Batch Normalization is applied after convolution to stabilize the distribution of intermediate feature values.

Its major advantages include

- Faster model convergence
- Reduced internal covariate shift
- Improved training stability
- Better generalization
- Reduced sensitivity to weight initialization

By normalizing the outputs of intermediate layers, Batch Normalization enables the network to learn efficiently even with deeper architectures.

7.7 Max Pooling Layer

Pooling layers reduce the spatial dimensions of feature maps while preserving the most important information.

The proposed CNN uses Max Pooling with a pooling window of

$$2 \times 2$$

The maximum value from each pooling window is selected.

For example,

$$\begin{bmatrix} 1 & 3 \\ 5 & 2 \end{bmatrix} \rightarrow 5$$

This significantly reduces the computational complexity of the model.

The benefits of Max Pooling are listed below.

- Reduces computation.
- Reduces memory usage.
- Minimizes overfitting.
- Provides translation invariance.
- Retains important image features.

After the pooling operation, the extracted feature maps become smaller while retaining the most discriminative information required for handwritten digit classification.

7.8 Dropout Layer

One of the major challenges in deep learning is overfitting. Overfitting occurs when the neural network performs extremely well on the training dataset but fails to generalize to unseen testing data. To overcome this problem, the proposed CNN incorporates Dropout layers.

Dropout randomly deactivates a certain percentage of neurons during each training iteration. As a result, the network cannot become overly dependent on a particular neuron and is forced to learn more generalized features.

In this project, a dropout rate of approximately 0.25 to 0.50 is used after selected convolutional and fully connected layers.

The major advantages of Dropout are:

- Reduces overfitting.
- Improves model generalization.
- Prevents co-adaptation of neurons.
- Produces more robust feature representations.
- Improves testing accuracy.

7.9 Flatten Layer

After passing through multiple convolutional and pooling layers, the extracted feature maps are still represented as multidimensional matrices. However, the fully connected neural network expects one-dimensional input.

Therefore, a Flatten layer is used to convert the feature maps into a one-dimensional feature vector.

For example,

$$7 \times 7 \times 64$$

is transformed into

$$3136$$

features.

The Flatten layer does not modify the learned information; it simply changes the data representation so that it can be processed by the dense layers.

7.10 Fully Connected Layers

The fully connected layers are responsible for performing the final classification based on the features extracted by the convolutional layers.

Every neuron in a dense layer is connected to every neuron in the previous layer. These layers learn complex nonlinear relationships among the extracted features and gradually improve the classification capability of the network.

The proposed CNN uses one or more dense layers with ReLU activation before the final output layer.

The advantages of fully connected layers include:

- High learning capability.

- Better feature combination.
- Improved decision making.
- Higher classification accuracy.

7.11 Output Layer

The final layer of the proposed CNN is the Softmax output layer. Since handwritten digit recognition is a multiclass classification problem, the Softmax function converts the outputs of the dense layer into probability values.

The probability for each output class is computed as

$$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}}$$

where

- z_i represents the output score for class i .
- $P(y_i)$ represents the predicted probability.

The class having the highest probability is selected as the final prediction. For example,

$$[0.01, 0.02, 0.95, 0.00, \dots]$$

indicates that the predicted digit is

2

with a confidence of 95%.

7.12 Layer-wise Architecture

Table 7.1 summarizes the CNN architecture used in this project.

Table 7.1: Layer-wise CNN Architecture

Layer	Purpose	Activation
Input Layer	Receives image	–
Conv2D (32 Filters)	Feature extraction	ReLU
Batch Normalization	Normalize features	–
MaxPooling2D	Down sampling	–
Conv2D (64 Filters)	Deep feature extraction	ReLU
Batch Normalization	Stable learning	–
MaxPooling2D	Dimension reduction	–
Dropout	Prevent overfitting	–
Flatten	Convert to vector	–
Dense (128)	Feature learning	ReLU
Dropout	Regularization	–
Dense (10)	Classification	Softmax

The architecture provides an effective balance between computational efficiency and classification accuracy, making it highly suitable for handwritten digit recognition using the MNIST dataset. This CNN architecture forms the core of the proposed system and enables accurate real-time digit prediction through the deployed Streamlit application.

Methodology and Algorithms

The methodology describes the systematic procedure followed to develop the proposed handwritten digit recognition system. The system is implemented using a Convolutional Neural Network (CNN) trained on the MNIST dataset. The complete workflow includes data collection, preprocessing, CNN model development, training, evaluation, and deployment through a Streamlit web application. Figure 8.1 illustrates the overall methodology of the proposed system.

8.1 Proposed System

The proposed system is designed to recognize handwritten digits accurately by utilizing the feature learning capability of Convolutional Neural Networks. Instead of manually extracting image features, the CNN automatically learns relevant features during training. After training, the model is integrated with a Streamlit application, enabling users to draw handwritten digits and obtain real-time predictions.

The overall development process consists of the following stages:

1. Dataset Collection
2. Data Preprocessing
3. CNN Model Development
4. Model Training
5. Performance Evaluation
6. Model Deployment

8.2 Overall Workflow

The overall workflow of the proposed handwritten digit recognition system is shown in Figure 8.1. The workflow illustrates the sequence of operations from dataset acquisition

to real-time prediction.

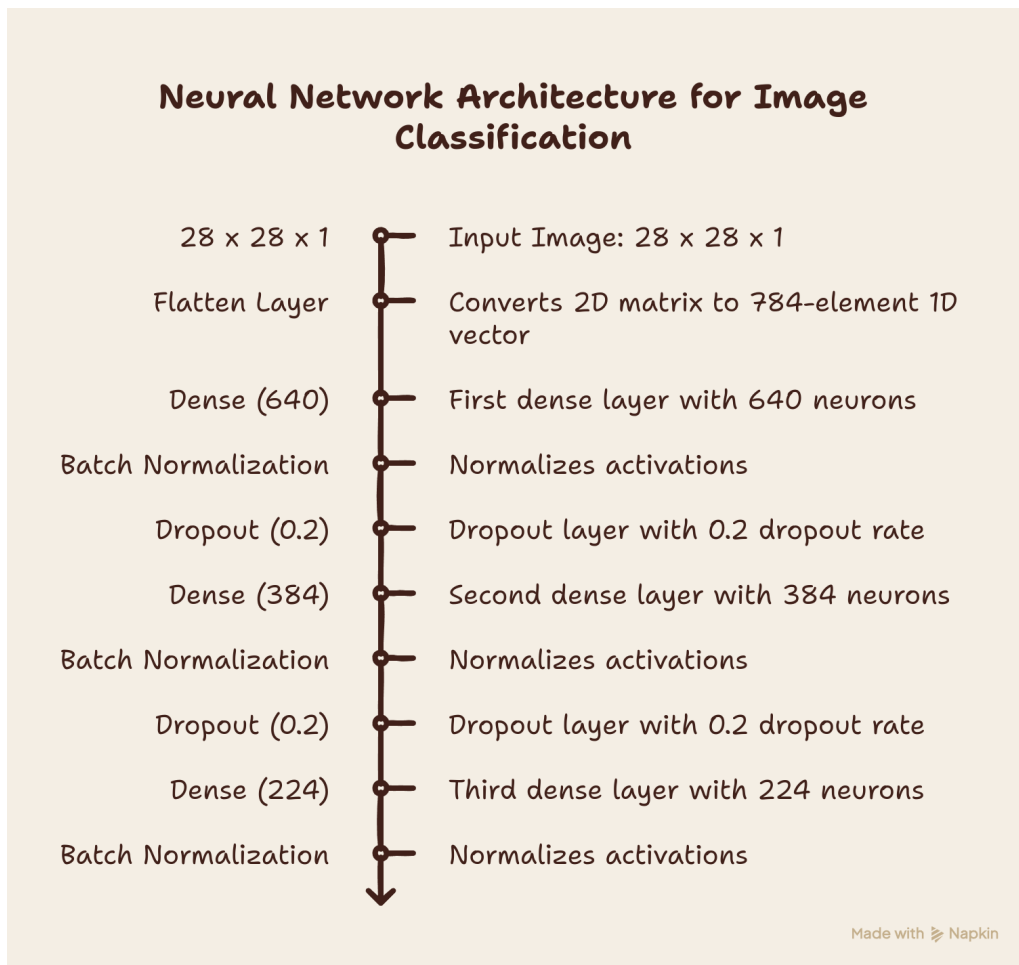


Figure 8.1: Overall Workflow of the Proposed System

The workflow begins with loading the MNIST dataset. The images are preprocessed and then used to train the CNN model. After successful training, the model is evaluated using test data and finally deployed through a Streamlit interface for real-time handwritten digit recognition.

8.3 Dataset Collection

The proposed system uses the MNIST handwritten digit dataset, which is one of the most widely accepted benchmark datasets for image classification. The dataset contains grayscale images of handwritten digits ranging from 0 to 9.

The dataset consists of:

- 60,000 training images
- 10,000 testing images

- Image size of 28×28 pixels
- 10 output classes representing digits 0–9

The large number of labeled images enables the CNN model to learn diverse handwriting styles and achieve high classification accuracy.

8.4 Data Preprocessing

Data preprocessing is an essential step before training the CNN model. Proper preprocessing improves learning efficiency, reduces computational complexity, and ensures consistency between training and testing data.

The preprocessing operations performed in this project are described below.

8.4.1 Normalization

The pixel values of MNIST images range from 0 to 255. To improve numerical stability during training, the pixel values are normalized to the range 0–1 using

$$x = \frac{x}{255}$$

where x represents the original pixel value.

Normalization accelerates convergence and improves model performance.

8.4.2 Image Reshaping

Each image is reshaped from

$$28 \times 28$$

to

$$28 \times 28 \times 1$$

to include the grayscale channel required by the convolutional layers.

8.4.3 Label Encoding

The output labels representing digits 0–9 are converted into one-hot encoded vectors using categorical encoding. This representation allows the CNN model to perform multi-class classification efficiently.

8.4.4 Training and Testing Split

The MNIST dataset is already divided into separate training and testing sets. The training dataset is used for learning the model parameters, while the testing dataset is used to evaluate the performance of the trained model on previously unseen data.

8.5 CNN Model Development

After preprocessing, the CNN model is constructed using the TensorFlow and Keras frameworks. The Sequential API is used to build the architecture by stacking multiple layers.

The architecture consists of:

- Convolutional Layers
- Batch Normalization Layers
- Max Pooling Layers
- Dropout Layers
- Flatten Layer
- Fully Connected Dense Layers
- Softmax Output Layer

Each convolutional layer extracts meaningful image features, while pooling layers reduce the spatial dimensions of feature maps. Batch normalization stabilizes training, dropout minimizes overfitting, and the dense layers perform the final classification. The Softmax layer predicts the probability of each handwritten digit class.

The architecture is trained using the Adam optimization algorithm with categorical cross-entropy as the loss function. During training, the model gradually learns the distinguishing features of handwritten digits, enabling accurate classification of unseen images.

8.6 Algorithms Used

The proposed handwritten digit recognition system employs several deep learning algorithms and techniques that work together to achieve accurate classification. Each algorithm performs a specific function during feature extraction, training, and prediction.

8.6.1 Convolution

The convolution operation is responsible for extracting important visual features such as edges, curves, and strokes from the input image. Small learnable filters slide across the image to generate feature maps.

The convolution operation is represented as

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) \times K(m, n)$$

where:

- X is the input image.
- K is the convolution kernel.
- Y is the generated feature map.

Multiple convolution layers enable the CNN to learn increasingly complex image features.

8.6.2 ReLU Activation Function

After each convolution operation, the Rectified Linear Unit (ReLU) activation function is applied to introduce non-linearity into the network.

$$f(x) = \max(0, x)$$

ReLU improves training speed, reduces the vanishing gradient problem, and enables efficient learning of complex patterns.

8.6.3 Max Pooling

Max Pooling reduces the spatial dimensions of feature maps while preserving the most significant information.

A pooling window of

$$2 \times 2$$

is used to select the maximum value from each region. This reduces computational cost and improves generalization.

8.6.4 Batch Normalization

Batch Normalization normalizes intermediate feature values during training. It accelerates convergence, stabilizes learning, and reduces sensitivity to weight initialization, leading to better overall performance.

8.6.5 Dropout

Dropout is used to reduce overfitting by randomly deactivating a fraction of neurons during training. This forces the network to learn more generalized features and improves prediction performance on unseen data.

8.6.6 Adam Optimizer

The Adam optimizer is used to update network weights efficiently during training. It combines the advantages of momentum and adaptive learning rates, resulting in faster convergence and improved optimization.

8.6.7 Softmax Classifier

The output layer uses the Softmax activation function to convert the network outputs into probability values for the ten digit classes.

$$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}}$$

The digit with the highest probability is selected as the final prediction.

8.7 Model Training

The CNN model is trained using the preprocessed MNIST training dataset. During training, batches of images are passed through the network, predictions are generated, and the loss is calculated using the categorical cross-entropy loss function. The Adam optimizer updates the network weights iteratively until the model converges.

Training is performed for multiple epochs, allowing the model to gradually learn distinguishing features of handwritten digits. The training and validation accuracy are monitored throughout the process to evaluate learning progress and detect overfitting.

8.8 Model Evaluation

After training, the model is evaluated using the unseen MNIST test dataset. Several performance metrics are used to measure the effectiveness of the proposed system.

The evaluation includes:

- Training Accuracy
- Validation Accuracy
- Training Loss
- Validation Loss
- Confusion Matrix
- Classification Accuracy

These metrics provide a comprehensive assessment of the model's prediction capability and generalization performance.

8.9 Model Deployment

To provide an interactive user experience, the trained CNN model is deployed using the Streamlit framework. The web application allows users to draw handwritten digits on a digital canvas or upload an image for prediction.

Before prediction, the input image undergoes preprocessing steps such as grayscale conversion, resizing, normalization, and centering to match the format of the MNIST dataset. The processed image is then passed to the trained CNN model, which predicts the corresponding digit along with its confidence score.

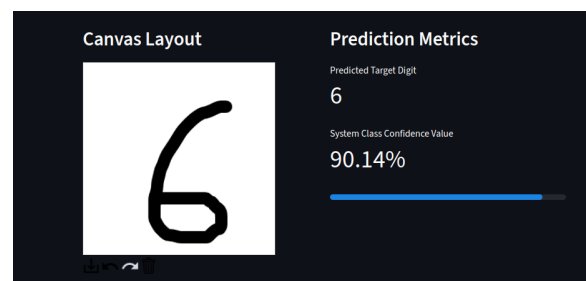
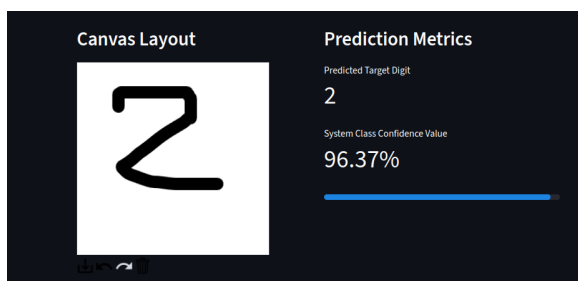


Figure 8.3 illustrates the deployment workflow.

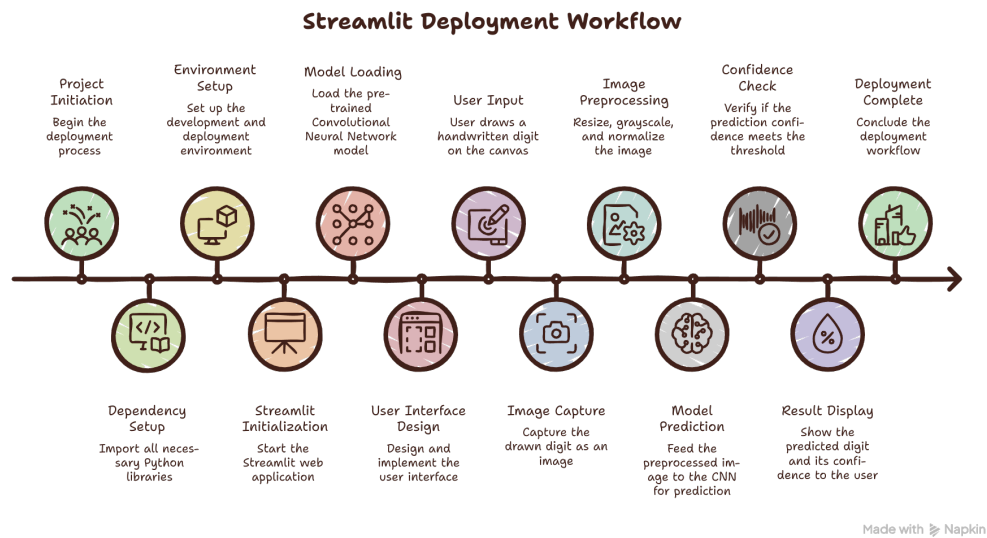


Figure 8.3: Model Deployment using Streamlit

8.10 Summary

The proposed methodology combines data preprocessing, CNN-based feature extraction, model training, evaluation, and deployment into a unified handwritten digit recognition system. The use of convolution, ReLU activation, max pooling, batch normalization, dropout, Adam optimization, and Softmax classification enables the model to achieve accurate and reliable recognition. Finally, deployment through a Streamlit web application provides an intuitive interface for real-time handwritten digit prediction, demonstrating the practical applicability of deep learning techniques in image classification.

Results

9.1 Training and Validation Accuracy

Figure 9.1 illustrates the training and validation accuracy of the proposed CNN model over multiple training epochs.

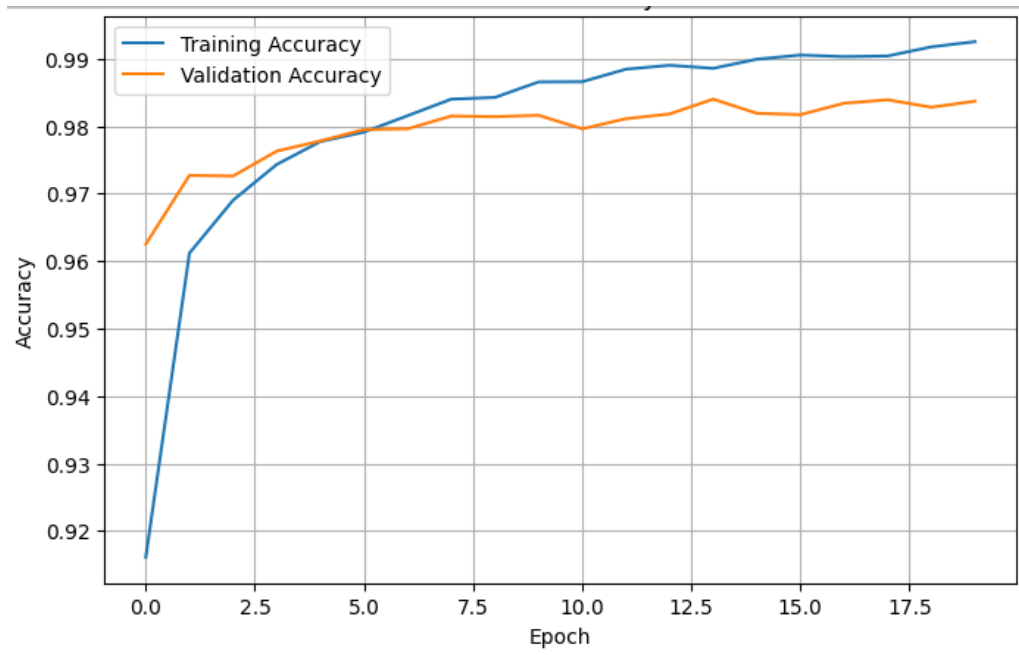


Figure 9.1: Training and Validation Accuracy

The training and validation accuracy curves demonstrate the learning performance of the proposed CNN model during the training process. Initially, the training accuracy increases rapidly from approximately 91.6% to above 98%, indicating that the model quickly learns the fundamental features of handwritten digits. As training progresses, both curves gradually stabilize, showing consistent improvement.

The final training accuracy reaches approximately 99.2%, while the validation accuracy stabilizes around 98.3%. The small difference between the two curves indicates that the model generalizes well to unseen data and does not suffer from significant overfitting.

The absence of large fluctuations in the validation curve also demonstrates stable learning and effective optimization.

Overall, the accuracy graph confirms that the proposed CNN architecture successfully learns discriminative features from the MNIST dataset and achieves excellent classification performance.

9.2 Training and Validation Loss

Figure 9.2 illustrates the variation of training and validation loss during the training process.

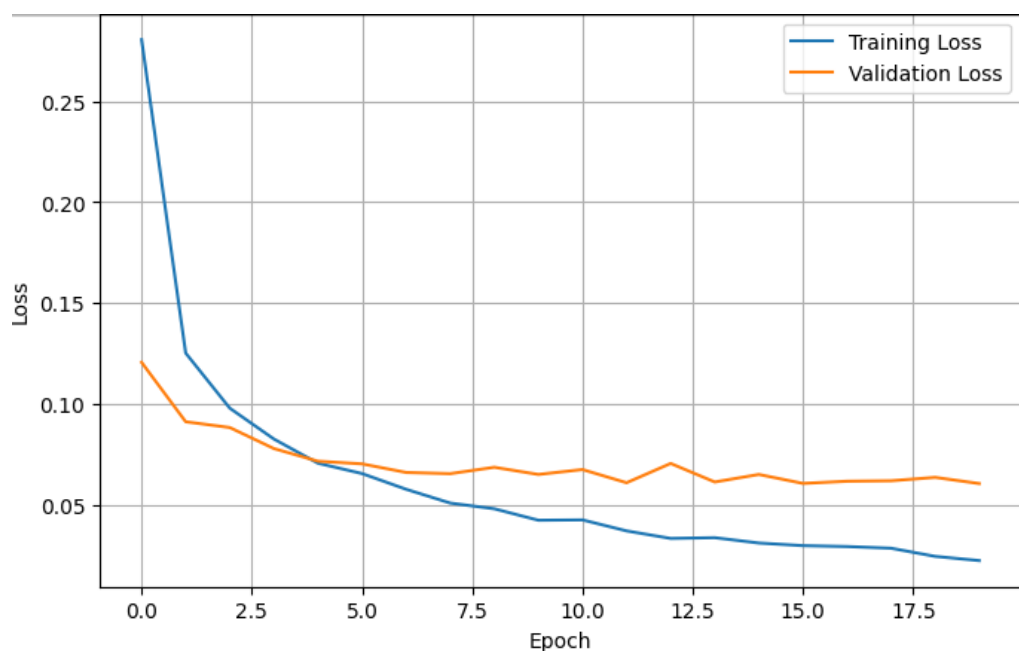


Figure 9.2: Training and Validation LOss

The training and validation loss curves indicate how the prediction error decreases as the CNN model learns from the MNIST dataset. At the beginning of training, the training loss is relatively high because the model parameters are randomly initialized. As the epochs progress, the loss decreases rapidly, indicating that the model is effectively learning meaningful image features.

The validation loss also decreases steadily and remains close to the training loss throughout the training process. The small gap between the two curves suggests that the model generalizes well to unseen data and does not suffer from significant overfitting. Minor fluctuations in the validation loss are expected because the model is evaluated on previously unseen samples after each epoch.

By the final epoch, the training loss reaches approximately **0.02**, while the validation loss stabilizes around **0.06**. The smooth convergence of both curves demonstrates stable learning and confirms that the selected CNN architecture, together with the Adam

optimizer, successfully minimizes the classification error while maintaining good generalization capability.

9.3 Confusion Matrix

Figure 9.3 presents the confusion matrix obtained by evaluating the trained CNN model on the MNIST test dataset.

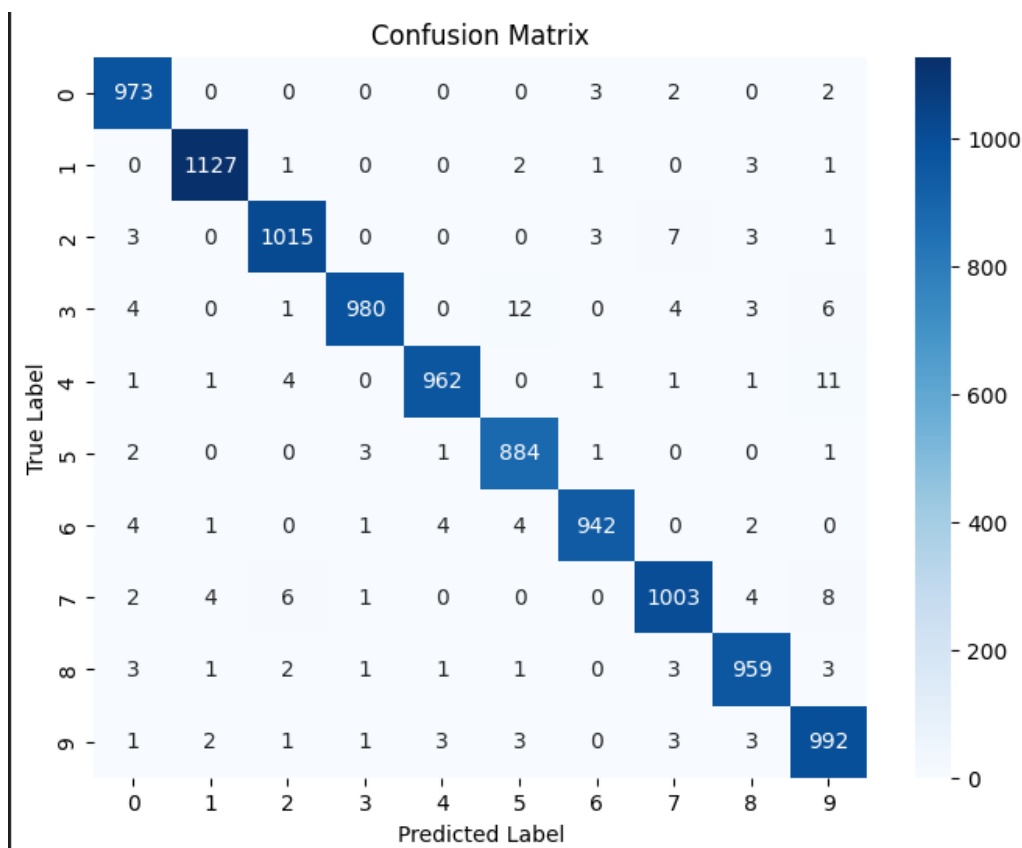


Figure 9.3: Confusion Matrix

The confusion matrix provides a detailed analysis of the classification performance for each handwritten digit. The rows represent the true digit labels, while the columns represent the predicted labels. The diagonal elements correspond to correctly classified samples, whereas the off-diagonal elements represent misclassifications.

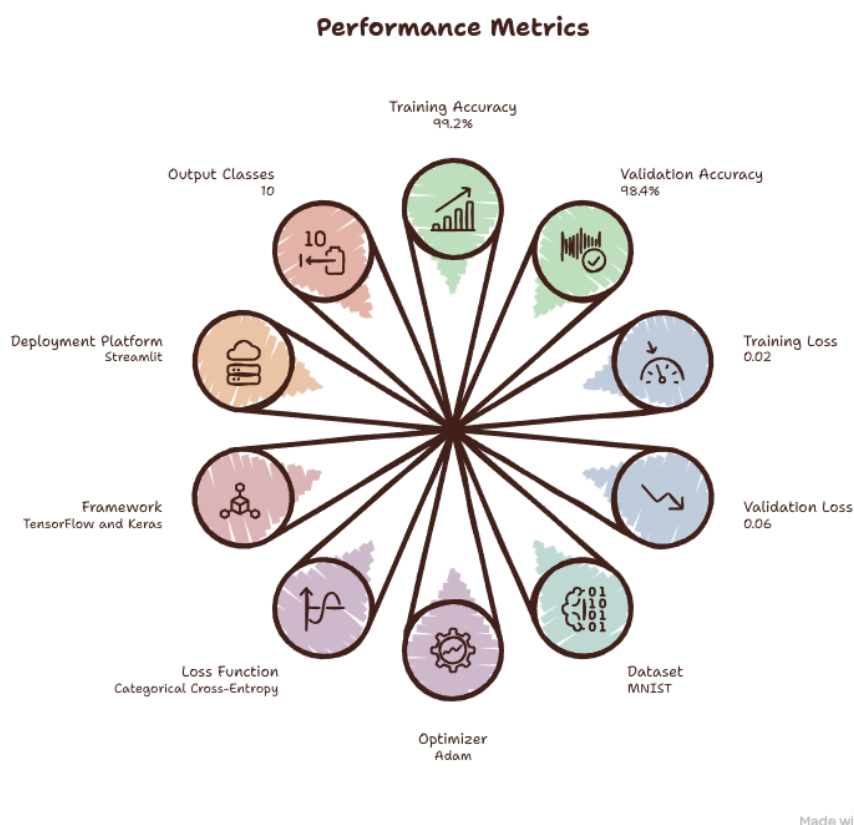
It can be observed that almost all values are concentrated along the main diagonal, indicating that the CNN model correctly classifies the majority of handwritten digits. Only a few samples are misclassified, mainly between digits that have similar handwritten shapes, such as **3 and 5**, **4 and 9**, or **7 and 9**. Such errors are expected because different writers often produce visually similar digit patterns.

The confusion matrix demonstrates the high reliability and robustness of the proposed CNN model. The large number of correct predictions and the very small number

of misclassifications confirm that the model has learned discriminative image features effectively and performs well on previously unseen handwritten digit images.

9.4 Performance Summary

Figure summarizes the overall performance of the proposed handwritten digit recognition system.



The experimental results demonstrate that the proposed CNN model achieves excellent performance on the MNIST handwritten digit dataset. The high training and validation accuracy, together with the low loss values and strong confusion matrix, indicate that the model is capable of accurately recognizing handwritten digits while maintaining good generalization on unseen data. These results validate the effectiveness of Convolutional Neural Networks for handwritten digit recognition and confirm the suitability of the proposed system for real-time image classification applications.

Applications and Future Scope

10.1 Applications

The proposed handwritten digit recognition system has several practical applications in various domains where automatic interpretation of handwritten information is required. Some of the important applications are listed below:

- **Banking Systems:** Used for automatic recognition of handwritten digits on bank cheques, deposit slips, and financial documents.
- **Postal Services:** Helps in recognizing handwritten postal codes for automatic mail sorting and delivery.
- **Document Digitization:** Assists in converting handwritten records into digital format for easy storage and retrieval.
- **Educational Systems:** Can be used for automatic evaluation of handwritten answer sheets and examination forms.
- **Optical Character Recognition (OCR):** Serves as a fundamental component in OCR systems for recognizing handwritten numerical data.
- **Research and Computer Vision:** Acts as a benchmark application for studying deep learning, image processing, and pattern recognition techniques.

10.2 Future Scope

Although the proposed system achieves high accuracy on the MNIST dataset, several improvements can be made to enhance its capabilities in future. The model can be extended to recognize handwritten alphabets, mathematical symbols, and multilingual characters. More advanced deep learning architectures such as ResNet or EfficientNet may be employed to improve recognition accuracy on complex datasets.

The system can also be deployed as a mobile or cloud-based application to enable real-time handwritten digit recognition on portable devices. Integration with complete Optical Character Recognition (OCR) systems would allow automatic processing of handwritten documents, forms, and historical records. These enhancements would make the proposed system suitable for a wider range of real-world applications and large-scale intelligent document processing.

Conclusion

This project successfully developed a handwritten digit recognition system using Convolutional Neural Networks (CNNs). The MNIST handwritten digit dataset was used to train and evaluate the model, enabling it to recognize handwritten digits from 0 to 9 with high accuracy. Appropriate preprocessing techniques, including normalization, image reshaping, and one-hot encoding, were applied to improve the learning efficiency of the model.

A CNN architecture consisting of convolutional, pooling, batch normalization, dropout, and dense layers was implemented using TensorFlow and Keras. The model was trained using the Adam optimizer and categorical cross-entropy loss function, resulting in excellent classification performance. The training and validation accuracy curves, loss graphs, and confusion matrix demonstrate that the proposed model learns effectively and generalizes well to unseen handwritten digit images.

To enhance usability, the trained CNN model was deployed using the Streamlit framework, allowing users to draw handwritten digits and obtain real-time predictions with confidence scores. This demonstrates the practical application of deep learning techniques in developing interactive image classification systems.

Overall, the objectives of the project were successfully achieved. The proposed system provides an accurate, efficient, and user-friendly solution for handwritten digit recognition while demonstrating the effectiveness of Convolutional Neural Networks in computer vision tasks. The project also offers a strong foundation for future research in Optical Character Recognition (OCR), handwritten character recognition, and other deep learning-based image classification applications.

References

1. Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-Based Learning Applied to Document Recognition,” *Proceedings of the IEEE*, Vol. 86, No. 11, pp. 2278–2324, 1998.

Available: <https://ieeexplore.ieee.org/document/726791>

2. Y. LeCun, C. Cortes, and C. J. C. Burges, “The MNIST Database of Handwritten Digits.”

Available: <https://yann.lecun.com/exdb/mnist/>

3. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.

Available: <https://www.deeplearningbook.org/>

4. A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” *Advances in Neural Information Processing Systems (NIPS)*, 2012.

Available: <https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks>
[pdf](#)

5. TensorFlow Developers, *TensorFlow Documentation*.

Available: <https://www.tensorflow.org/>

6. Keras Team, *Keras Documentation*.

Available: <https://keras.io/>

7. Streamlit Developers, *Streamlit Documentation*.

Available: <https://docs.streamlit.io/>